Examen 2: algoritmos

Demostraciones:

Problema 1:

1. Subestructura óptima:

por contradicción → hipotesis: sea A^* una solución óptima para el monto m, con $|A^*| = b$ monedas. Sea $c_1 \in A^*$ la primera moneda.

Decisiones:

- En cada paso:

  se elige una moneda $c \in \{1, 5, 10, 25\}$

  $c \leq m$

  moneda elegida = decisión

Subproblema:

- al elegir la moneda $(m-c)$: dar el cambio por el monto usando el menor # de monedas.

$A' = \frac{A^*}{c}$ $\quad A' = (m-c)$ (resuelve)

$|S'| = b-1$ monedas

△ Por contradicción A' no es óptimo así que existe A''

$|A''| < b-1$ $\quad A''$ resuelve $(m-c_1)$

$$(A'' \cup \{c_1\}) = |A''| + 1 < (b-1) + 1 = k$$

□ Por contradicción A' es una subestructura óptima.

2. Subproblemas traslapados:

Relación de recurrencia: best(m) = # min para m

best(0) = 0

best(m) = 1 + min (best(m-c) | $c \in \{1, 5, 10, 25\}$ $c \leq m$

Subproblemas traslapados: best(m) recursivamente
best(r) $r<m$ se evalua en varias ramas
hay $O(m)$ subproblemas pero se recalculan exponencialmente sin memoización.

Solución apoyada en la recurrencia:

$$dp[0] = 0$$
$$dp[j] = 1 + \min(dp[j-c] \mid c \in \{1, 5, 10, 25\},\ c \leq j)$$

complejidad: $O(m \cdot |c|)$ tiempo
$O(m)$ espacio

□ Cada subproblema se resuelve 1 vez.

3. Greedy-choice property:

elegir la moneda más cara (mayor denominación) sin exceder el monto, cualquier solución best se puede cambiar para que elija la moneda más grande.

Óptimo local.

Demostración: sea best "una solución óptima" para m

best	Intercambio	Efecto
5 monedas → 0.01	5 + 0.01 → 1 + 0.05	-4 monedas
2 monedas → 0.05	2 + 0.05 → 1 + 0.1	-1 moneda
3 monedas → 0.1	3 + 0.1 → 1 + 0.25 + 1 + 0.05	-1 moneda

□ Con los intercambios no se aumenta el total de monedas.

Como este problema tiene una subestructura óptima

y greedy choice, el algoritmo greedy que siempre elige la moneda más cara produce una solución óptima global.

4. Matroide ponderada:

las monedas disponibles son los objetos del problema, se usa un multiconjunto con instancias indexadad de cada denominación $A = \{s_1, s_2 \dots 10_1, 25_1\}$

Conjunto de objetos

Familia de Independientes

$D \subseteq A$

$I = \{D \subseteq A \mid \Sigma(c \in D)$ denominación$(c) \leq m\}$

□ Es independiente si la suma de sus denominaciones no excede m

Propiedad de Herencia:

si $D \in I$ y $B \subseteq D \rightarrow B \in I$

$\Sigma\{c \in B\}$ denominación $(c) \leq$ $\Sigma\{c \in D\}$ denominación $(c) \leq$ $m \rightarrow B \in I$

□ Todo subconjunto de un conjunto factible es factible

Propiedad de Intercambio:

Si $D, B \in I$ con $|D| < |B|$ existe $c \in B \setminus A$ tal que $A \cup \{c\} \in I$

$\Sigma\{c' \in D \cup \{c\}\}$ denominación $(c') = \Sigma\{c' \in D\}$ denominación (c') + denominación $(c) \leq m$ □ agregar una moneda de B no puede exceder m ya que B es independiente y todas las demonaciones son positivas la suma de D ya es $\leq m$.

función de peso w: $w(c) = 1$ para toda moneda c

minimizar cantidad de monedas es eq. a maximizar con peso negativo.

el algoritmo que en cada paso agrega el elemento de mayor deno-